Étude de cas 1 : Lecture automatique des chèques

Jalon 4 : Lire le montant comme une séquence

chapitre 4 - RNN, LSTM et GRU

M2 SDIA - Advanced Artificial Intelligence

Année universitaire 2026–2027

Objectif du jalon

Lire le montant comme une suite de chiffres de **longueur variable**, avec un encodeur-décodeur récurrent, et en tirer une confiance utilisable pour l'automatisation.

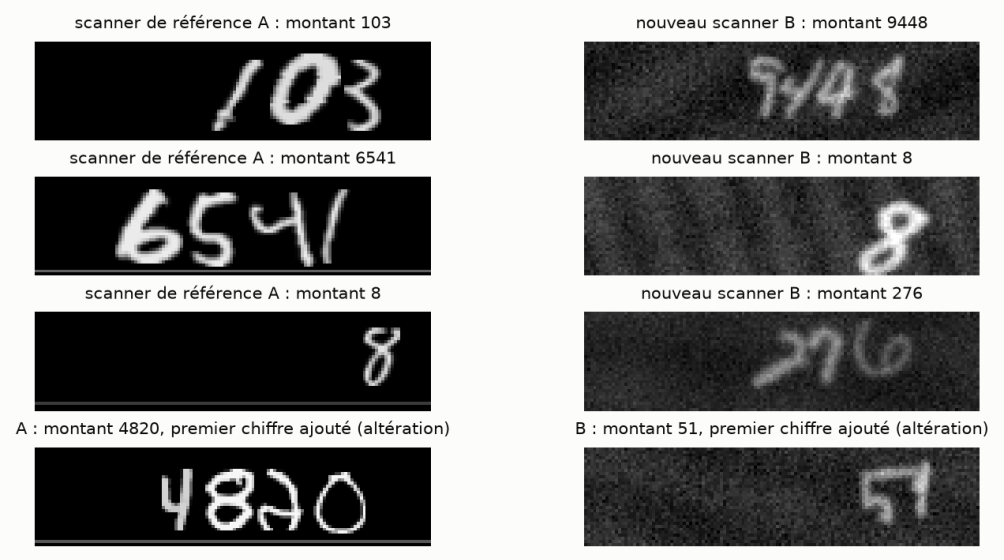
Jalon autonome : ce jalon ne suppose pas d'avoir terminé les jalons précédents. Il s'appuie uniquement sur le notebook de démarrage `uc1_demarrage.ipynb`, le dossier `data/` et le code de démarrage de la section 3. Vous pouvez repartir de votre propre travail antérieur, mais rien ne l'exige.
Environnement : Python 3, TensorFlow/Keras, scikit-learn ; Google Colab avec GPU recommandé.

1 Contexte

La banque fictive *Banque Atlas* traite chaque jour des dizaines de milliers de chèques. Le montant en chiffres, écrit à la main dans un champ cadré à droite, doit être lu automatiquement ; lorsque la machine n'est pas assez sûre d'elle, le chèque part vers un opérateur. La banque veut automatiser le plus possible **sans dépasser 1 % d'erreur** sur les chèques traités automatiquement.

Certains chèques sont **altérés** : un fraudeur ajoute un chiffre devant le montant ou retouche un chiffre existant, avec un tracé légèrement différent. Enfin, les agences reçoivent un **nouveau scanner** dont les images sont plus pâles, floues et portent un motif de sécurité ; seules 600 images de ce scanner sont étiquetées.

L'étude de cas en cinq jalons : un jalon par chapitre du cours, rendu après le chapitre correspondant, puis une synthèse défendue à l'oral en fin de module (note de décision, prédictions sur le test caché, soutenance).



2 Données et cibles de ce jalon

Fichier	Contenu
`uc1_domaineA.npz`	scanner de référence : 15 000 / 3 000 / 5 000 images (entraînement, validation, test)

Chaque image, en niveaux de gris, mesure 32×128 pixels (encre claire sur fond sombre). Elle est accompagnée du montant écrit (chaîne de 1 à 5 chiffres), d'un indicateur d'altération et du type d'altération. Les chiffres proviennent du jeu public MNIST ; les montants, leur mise en page, les altérations et les défauts du scanner sont simulés. Les ensembles sont imposés et n'ont aucune image de chiffre en commun.

Cibles du cahier des charges concernées

Exigence	Mesure (ensemble de test)	Cible
Lire le montant (scanner A)	part des montants entièrement exacts	$\geq 0{,}85$
Automatiser (scanner A)	part des chèques traités automatiquement avec au moins 99 % de lectures exactes	$\geq 0{,}40$
Déploiement	taille du modèle ; latence par chèque	≤ 5 Mo ; ≤ 50 ms

Le **taux d'automatisation** se calcule en triant les chèques par confiance décroissante : c'est la plus grande part traitée automatiquement telle qu'au moins 99 % de ces lectures soient exactes (fonction `taux_automatisation` du notebook). Ces cibles décrivent le système *final* : elles ne sont pas exigées à la fin de ce jalon. Mesurez-vous par rapport à elles et expliquez l'écart ; un écart argumenté vaut mieux qu'une cible atteinte sans explication.

3 Point de départ

Le notebook de démarrage fournit l'encodage en séquence (section 2) : jetons 0 à 9 pour les chiffres, 10 pour le remplissage, 11 pour le début, 12 pour la fin, avec la fonction `vers_sequence`

qui produit l'entrée du décodeur, la cible décalée d'un pas et le masque.

Le code ci-dessous construit le tronc convolutif, transforme la carte de caractéristiques en suite de colonnes et l'encode par un GRU bidirectionnel. **Le décodeur est à écrire : c'est le cœur de ce jalon.**

Code de démarrage (fichier `uc1_jalon4_depart.py`)

```
# Étude de cas 1 - Jalon 4 : code de démarrage
# À exécuter à la suite du notebook de démarrage, exécuté en entier.

# 1. Cibles en séquence (section 2 du notebook) : entrée du décodeur
#    (forçage par l'enseignant), cible décalée d'un pas, masque
SEQ = {s: vers_sequence(A["montant_" + s]) for s in ["train", "val", "test"]}
entree_dec, cible_dec, masque_dec = SEQ["train"]
print(A["montant_train"][0], entree_dec[0], cible_dec[0])

# 2. Tronc convolutif : trois blocs convolution 3x3 + max-pooling ;
#    le dernier pooling ne réduit que la hauteur
def tronc_cnn(image):
    x = image
    for filtres, pool in ((32, (2, 2)), (64, (2, 2)), (64, (2, 1))):
        x = layers.Conv2D(filtres, 3, padding="same", activation="relu")(x)
        x = layers.MaxPooling2D(pool)(x)
    return x

# 3. Carte de caractéristiques -> suite de colonnes, de gauche à droite,
#    encodée par un GRU bidirectionnel
image = keras.Input((32, 128, 1), name="image")
carte = tronc_cnn(image)
h, w, c = carte.shape[1:]
colonnes = layers.Reshape((w, h * c))(layers.Permute((2, 1, 3))(carte))
etats = layers.Bidirectional(layers.GRU(128, return_sequences=True))(colonnes)
encodeur = keras.Model(image, etats, name="encodeur")

# 4. À compléter : un décodeur GRU qui reçoit les jetons précédents
#    (Embedding(13, 32)), part d'un état initial tiré de l'encodeur et
#    prédit le jeton suivant (softmax à 13 classes). Entraînement :
#    fit([images, entree_dec], cible_dec, sample_weight=masque_dec)
```

4 Travail demandé

Jalon 4 : Lire le montant comme une séquence

Notions mobilisées : encodeur-décodeur, forme many-to-many, forçage par l'enseignant, jetons de début et de fin, décodage glouton, LSTM, GRU, réseaux bidirectionnels.

1. Écrire le décodeur récurrent (plongement du jeton précédent, état initial issu de l'encodeur, softmax à 13 jetons) et l'entraîner par forçage par l'enseignant, en pondérant la perte par le masque.
2. Implémenter le décodage glouton et une confiance adaptée (produit des probabilités jusqu'au jeton de fin) ; calculer l'exactitude du montant et le taux d'automatisation.

3. Comparer LSTM et GRU à budget de paramètres comparable, puis comparer la lecture en séquence à une lecture par positions construite sur le même tronc (aplatissement, couche dense, cinq têtes), en fonction de la longueur du montant.
4. Analyser les erreurs : le décodeur s'arrête-t-il au bon moment ? Que se passe-t-il sur les montants les plus longs, et sur les chèques altérés ?

Vérification théorique (à la main, puis vérifiée par le code)

1. Pour le tronc convolutif du code de démarrage (trois blocs, le dernier pooling ne réduisant que la hauteur), combien de pas de temps l'encodeur reçoit-il, et quelle largeur d'image chaque pas représente-t-il ?
2. Combien de paramètres compte un GRU de 128 unités recevant des plongements de dimension 32 (formulation de Keras, avec `reset_after=True`) ? Vérifiez avec `summary()`.

À justifier dans votre page de décisions

Ce que la lecture en séquence permet et que la lecture par positions ne permet pas ; le choix entre LSTM et GRU ; l'effet du décodage glouton sur la latence.

5 À rendre à la fin du chapitre

Livrables du jalon 4

— le notebook du jalon, exécuté et reproductible (graine fixée, versions indiquées) ;
— le journal d'expériences à jour (fonction `journaliser` du notebook de démarrage, avec `jalon=4`), y compris les essais infructueux ;
— une page de décisions : tableau des résultats, choix retenus et justifications, réponses aux vérifications théoriques.

Protocole : tous les réglages se font sur la validation ; l'ensemble de test ne sert qu'une fois, à la fin du jalon, pour le modèle retenu. Ce jalon alimente la note de décision finale et la soutenance.